

* Hashing & File Structure

- It is an effective way to store the elements in some data structure.
- It allows us to reduce the number of comparisons.
- Using Hashing we can obtain the direct address of the record.

* Hash-Table :-

- Hash-Table is a data structure used for storing & retrieving data very quickly.
- Insertion of data in hash table is based on key value.
- Every Entry in Hash Table is associated with some key.
- Using a Hash key the required piece of data can be searched in the hash table by few or more comparisons.

* Hash-Function :-

- A Hash function is a mathematical formula which when applied to a key it produces an integer which can be used as an index for the key in Hash-Table.
- The main aim of Hash function is that elements should be relatively, randomly & uniformly distributed.
- It produces a unique set of integers within some suitable range in order to reduce the number of collisions.
- In practice there is no hash function that eliminates

Collision completely.

→ A good Hash fn can only minimize the number of collision by spreading the elements uniformly throughout the array.

* Different Hash Function:-

→ 1) Division Method:-

→ It is the most simple method of hashing an integer.

→ This method divides x by M and then uses the remainder obtained.

→ In this case Hash function can be given as

$$h(x) = x \bmod M$$

$$h(1234) = 1234 \bmod 100 \\ = 34$$

$$h(5642) = 5642 \bmod 100 \\ = 42$$

0	
34	1234
42	5642
99	

2) Multiplication Method:-

Step-1 Choose a constant A such that $0 < A < 1$

[recond]
Step-2 Multiply the key k by A .

Step-3 Extract the fractional part of kA

Step-4 Multiply the result of step-3 by size of Hash Table.

⇒ Table size = 1000

$K = 12345$

$A = 0.618033$

$KA = 7629.617385$

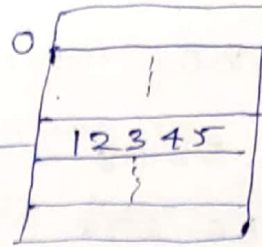
Step-4 0.617385×1000

$=$

$= 617.4$

≈ 617

999



3] Mid Square Method :-

→ Step 1:- Square the value of key i.e. k^2

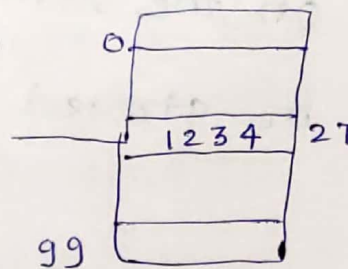
Step 2:- Extract the middle r digit of the result obtained in Step 1.

(r depends on the size of the table)

$K = 1234$

$K^2 = 15232756$

$\downarrow$
27



4) Folding Method :-

Step-1 :- Divide the key value into a number of parts like divide k into parts $k_1, k_2, \dots, k_n$ where each part has the same number of digit except last part which may have lesser digit than other parts.

Step-2 :- Add the individual part i.e. Obtain the sum of $k_1 + k_2 + \dots + k_n$. The hash value is produced by ignoring the last carry if any.

Note that the number of digit in each part of the key will vary depending upon the size of hash table.

e.g. Tablesize = 100

<u>Keys</u>	5678	321	34567
<u>Parts</u>	56 & 78	32 & 1	34 & 56 & 7
<u>Sum</u>	134	33	97
<u>Hashkey</u>	34	33	97

* Collision :-

→ Collision occurs when the Hash fn maps 2 different keys on the same location.

→ 2 records cannot be stored in the same location

e.g. Tablesize = 10

21, 35, 24, 17

key mod 10

	0
	1
	2
	3
24	4
35	5
	6
21	7
	8
	9

* Collision Resolution :-

→ A method used to solve problem of collision is known as collision resolution technique.

→ i] Open Addressing (Probing)

- Linear probing
- Quadratic probing
- Double Hashing
- Rehashing.

→ ii] Chaining.

* Linear probing :-

→ This is the simplest approach to resolve a collision.

→ In this technique if value is already stored at a location generated by $h(k)$ then following hash function is used to resolve the collision.

$$h(k, i) = [h'(k) + i] \bmod M$$

where M is size of hash table & $h'(k)$

& $h'(k) = k \bmod M$, i = The probe number that varies from 0 to $M-1$.

→ Therefore a given key k first the location generated by $(h'(k) \bmod M)$ is probed because first time $i=0$. If the location is free then value is stored in it else the second probe generate the address of the location given by $(h'(k) + i) \bmod M$

eg → Consider a hash table of size 10. Using linear probing insert the keys 72, 27, 36, 24, 63, 81, 92, 101

$$\{ h'(k) = k \bmod m \}$$

Soln:-

72

$$\begin{aligned} h(72, 0) &= (72 \bmod 10 + 0) \bmod M \\ &= (2 + 0) \bmod M \\ &= 2 \end{aligned}$$

	0
81	1
72	2
63	3
24	4
92	5
36	6
27	7
101	8
	9

$$\begin{aligned} h(27, 0) &= (27 \bmod 10 + 0) \bmod M \\ &= (7 + 0) \bmod M \\ &= 7 \end{aligned}$$

$$\begin{aligned} h(36, 0) &= (36 \bmod 10 + 0) \bmod M \\ &= (6 + 0) \bmod M \\ &= 6 \end{aligned}$$

$$\begin{aligned} h(24, 0) &= (24 \bmod 10 + 0) \bmod M \\ &= (4 + 0) \bmod M \\ &= 4 \end{aligned}$$

$$\begin{aligned} h(63, 0) &= (63 \bmod 10 + 0) \bmod M \\ &= (3 + 0) \bmod M \\ &= 3 \end{aligned}$$

$$\begin{aligned} h(81, 0) &= (81 \bmod 10 + 0) \bmod M \\ &= (1 + 0) \bmod M \\ &= 1 \end{aligned}$$

$$\begin{aligned} h(92, 0) &= (92 \bmod 10 + 1) \bmod m \\ &= 3 \bmod 10 \\ &= 5 \end{aligned}$$

$$h(101) = (1 + 1) \bmod 10$$

⇒ Quadratic Probing :

→ In this technique if a value is already stored at location generated by $h(k)$ then the following hash function is used to resolve the collision

$$h(k, i) = [h'(k) + (i)^2] \bmod M$$

e.g. 72, 27, 36, 24, 63, 81, 92, 101

$$h(92, 1) = (2 + 1) \bmod M$$

$$= 3 \bmod M$$

Already occupied

$$= (2 + 4) \bmod M$$

$$= 6 \bmod M$$

Already occupied

$$= (2 + 9) \bmod M$$

$$= 11 \bmod M$$

$$= 1$$

Already occupied

	0
81	1
72	2
63	3
24	4
101	5
36	6
27	7
92	8
	9

$$= (2 + 16) \bmod M$$

$$= 18 \bmod M$$

$$= 8$$

$$h(101, i) =$$

* Double Hashing :-

→ It is a technique in which a second hash fn is applied to a key when a collision occurs.

→ By applying a second hash fn you will get the number of position to be shifted from the point of collision.

$$H_1 = \text{key mod Table size } (m)$$

$$H_2 = M - (\text{key mod } m)$$

where M is a prime number smaller than Table size.

31, 90, 45, 22, 17, 49, 55

Table size = 10

$$M = 7$$

→ 17

$$\Rightarrow 17 \bmod 10$$

$$= 7$$

collision

$$\Rightarrow 7 - (17 \bmod 7)$$

$$= 7 - 3$$

$$= 4$$

90	0
17	1
22	2
	3
	4
45	5
55	6
31	7
	8
49	9

→ 55

$$= 5$$

collision

$$= 7 - (55 \bmod 7)$$

$$= 7 - 6$$

$$= 1$$

⇒ Rehashing :-

- In this technique, table size is resized.
- Size of table is doubled by creating a new table.
- It is preferable if total size of table is prime number.
- There are some situation in which rehashing is required.

- 1) When table is completely full.
- 2) With quadratic probing table is filled - half
- 3) In such situation we have to transfer entries from old table to new table by computing their position using suitable hash function.

37, 90, 55, 22, 17, 49, 87

Increase the table size to 20
But if prime number is preferable
∴ Table size = 23

Old Table

90	0
	1
22	2
	3
	4
55	5
	6
37	7
	8
	9

$$\begin{aligned}\rightarrow 37 \bmod 23 &= 14 \\ 90 \bmod 23 &= 21 \\ 55 \bmod 23 &= 9 \\ 22 \bmod 23 &= 22 \\ 17 \bmod 23 &= 17 \\ 49 \bmod 23 &= 3 \\ 87 \bmod 23 &= 18\end{aligned}$$

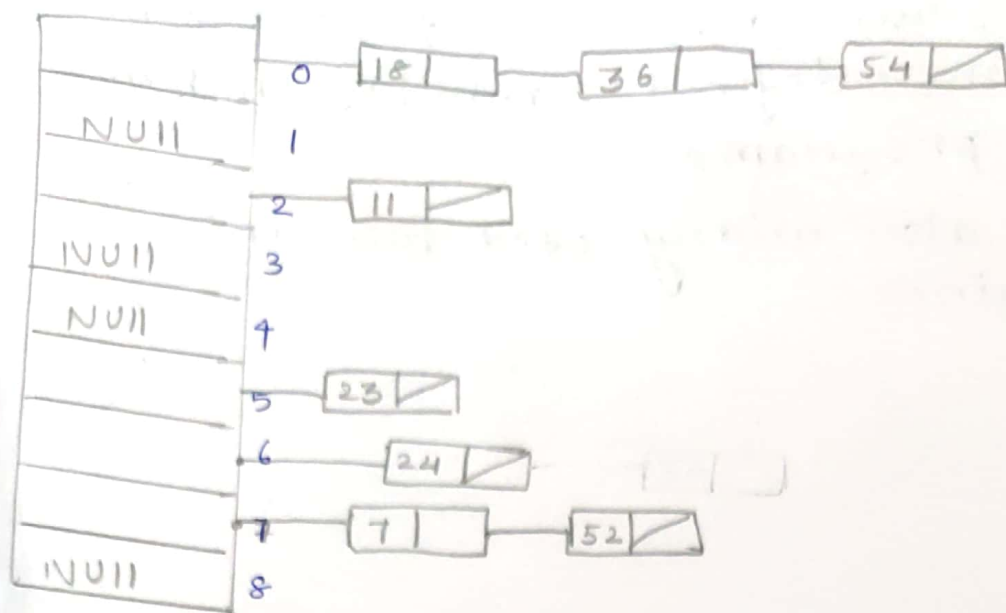
0	
3	49
9	55
14	37
17	17
18	87
21	90
22	22

Chaining :-

- In chaining each location in the hash table store a pointer to a link list that contains all the key values that were hashed to that location.
- It means location 1 in the hash table points to the Head of the link list of all the keys values that hashed to 1.
- However if no key value hashes to 1 then location 1 in the hash table contain NULL.
- Searching for the value in chain hash table is as simple as scanning a link list for an entry with the given key.
- Insertion operation appends the key to the end of link-list pointed by the hashed location.
- Deleting a key requires searching the list and remove the element.

e.g 37, 90, 55, 22, 17, 49, 87

Q Insert the keys 7, 24, 18, 52, 36, 54, 11, 23 in a chained hash table of 9 memory location
use $h(k) = k \bmod M$ $M=9$



⇒ Applications of Hashing :-

- Hash-table are widely used in situation where amount of data have to be accessed to quickly search & retrieve the information.
- Hashing is used for data-base indexing.
- Some Datab MS ^{store} a separate file known as the index file.
When data has to be retrieved from a file the key information first searched in the appropriate index file which references the exact record location of the data in the Data-base file.
- This key information in Index file is often stored as a hash value.
- Hashing technique is used to implement Compiler Symbol table in C++.

*) The compiler uses symbol table to keep a record of all the user defined symbols, which are used in C++ programs.

→ Hashing is also widely used for internet search engines.

Unit-4

Hashing & File Structure

What is File?

- A **file** is a **collection of records** where a record consists of one or more fields. Each contains the same sequence of fields.
- Each **field** is normally of **fixed length**.
- A sample file with four records is shown below:

Name	Roll No.	Year	Marks
AMIT	1000	1	82
KALPESH	1005	2	54
JITENDRA	1009	1	75
RAVI	1010	1	79

- There are **four records**
- There are four fields (Name, Roll No., Year, Marks)
- Records can be uniquely identified on the field 'Roll No.' Therefore, **Roll No. is the key field**.
- A database is a collection of files.

File Organizations

- **File Organizations**

1. Sequential files
2. Relative files
3. Direct files
4. Indexed Sequential files
5. Index files

Sequential Files

- It is the most common type of file.
- A **fixed format** is used for **record**.
- All **records** are of the **same length**.
- **Position** of each **field** in record and **length** of field is **fixed**.
- Records are **physically ordered** on the value of one of the fields - called the **ordering field**.

Block 1

Name	Roll No.	Year	Marks
AMIT	1000	1	82
KALPESH	1005	1	54
JITENDRA	1009	1	75
RAVI	1010	1	79

Block 2

Name	Roll No.	Year	Marks
RAMESH	1015	1	75
ROHIT	1025	1	65
JANAK	1026	1	75
AMAR	1029	1	79

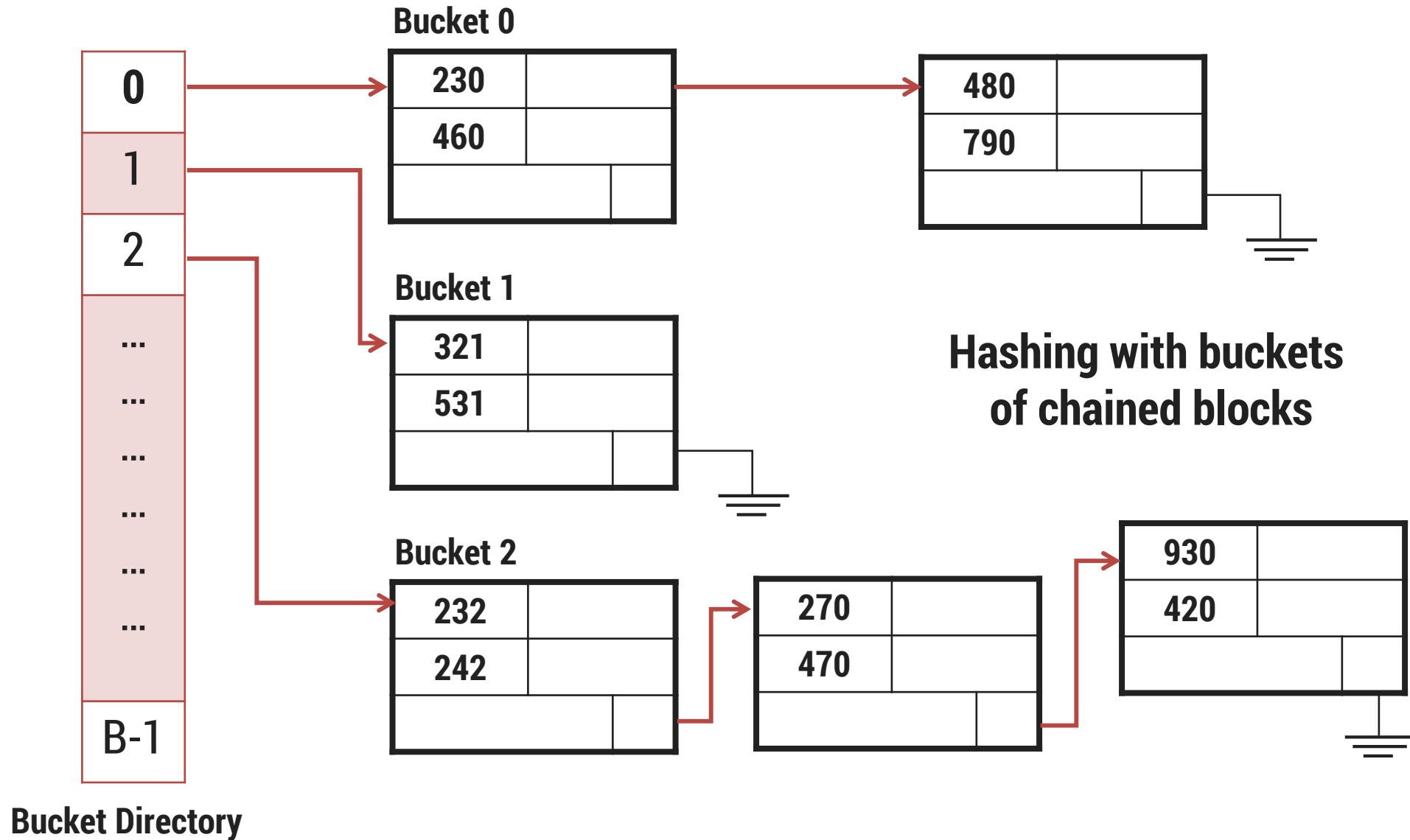
Advantages of Sequential Files

- Sequential files are **very simple** to manage.

Disadvantages of Sequential Files

- Sequential file **does not** give any **advantage** when the **search** operation is to be carried out **on non- ordering field**.
- **Inserting a record** is an **expensive operation**. Insertion of a new record requires finding of place of insertion and then all records ahead of it must be moved to create space for the record to be inserted. This could be very expensive for large files.
- **Deleting a record** is an **expensive operation**. Deletion too requires movement of records.
- **Modification of field value of ordering key** could be **time consuming**. Modifying the ordering field means the record can change its position. This requires deletion of the old record followed by insertion of the modified record.

Hashing (Direct file organization)



Hashing (Direct file organization)

- It is a common technique used for **fast accessing of records** on secondary storage.
- **Records** of a file are **divided among buckets**.
- A **bucket** is either **one disk block** or **cluster of contiguous blocks**.
- A **hashing function** maps a **key** into a **bucket number**. The buckets are numbered 0, 1, 2... $b-1$.
- A hash function **f** maps each **key** value into one of the integers **0 through $b - 1$** .
- If **x** is a **key**, **f(x)** is the number of **bucket** that contains the record **with key x**.
- The blocks making up each bucket could either be contiguous blocks or they can be chained together in a linked list.

Hashing (Direct file organization)

- Translation of bucket number to disk block address is done with the help of **bucket directory**. It **gives** the **address of the first block** of the chained blocks in a linked list.
- Hashing is quite efficient in retrieving a record on hashed key. The average number of block accesses for retrieving a record.

$$= 1 \text{ (bucket directory)} + \frac{\text{No of records}}{\text{No of buckets} \times \text{No of records per block}}$$

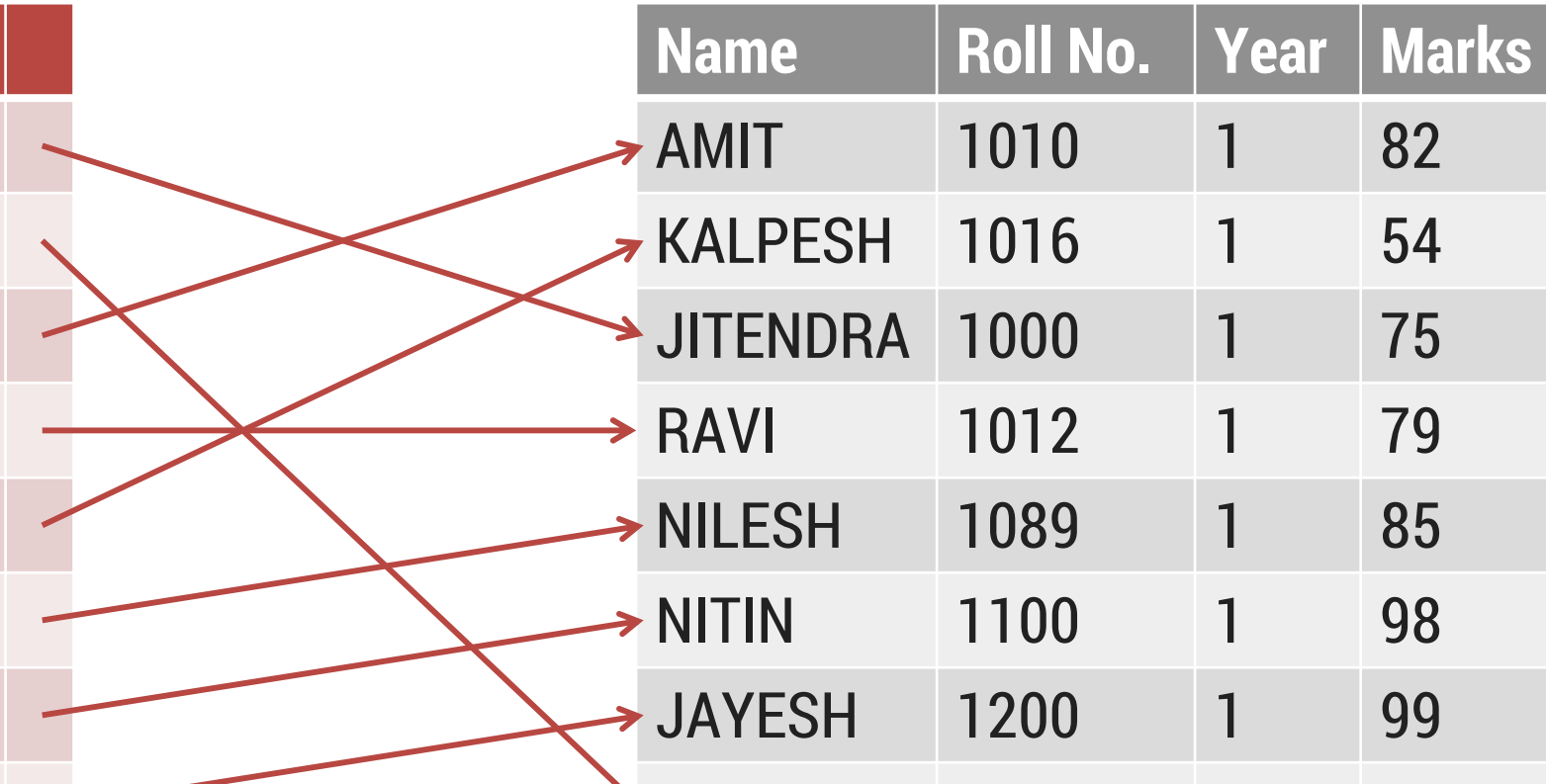
- Thus the **operation is b times faster** (b = number of buckets) than unordered file.
- To **insert a record with key value x** , the **new record** can added to the **last block** in the chain for bucket $f(x)$. If the record does not fit into the existing block, record is stored in a new block and this new block is added at the end of the chain for bucket $f(x)$.
- A well designed hashed structure requires two block accesses for most operations

Indexing

- **Indexing** is **used** to **speed** up **retrieval** of records.
- It is done with the help of a **separate sequential file**.
- **Each record** of in the **index file** consists of two fields, a key field and a pointer into the main file.
- To **find** a specific **record** for the given **key value**, **index** is **searched** for the given key value.
- **Binary search** can **used** to search in **index file**. After getting the address of record from index file, the record in main file can easily be retrieved.

Indexing

Index File		Main File			
Keyc		Name	Roll No.	Year	Marks
1000		AMIT	1010	1	82
1009		KALPESH	1016	1	54
1010		JITENDRA	1000	1	75
1012		RAVI	1012	1	79
1016		NILESH	1089	1	85
1089		NITIN	1100	1	98
1100		JAYESH	1200	1	99
1200		UMESH	1009	1	74



The diagram illustrates the indexing process. The Index File contains keys (1000, 1009, 1010, 1012, 1016, 1089, 1100, 1200) which are sorted in ascending order of Roll No. Each key points via a red arrow to a specific record in the Main File. The Main File records are not sorted by Roll No. For example, the key 1000 points to the record for AMIT (Roll No. 1010), and the key 1200 points to the record for UMESH (Roll No. 1009).

Index file is ordered on the ordering key Roll No. each record of index file points to the corresponding record. Main file is not sorted.

Advantages of Indexing

- Sequential file can be searched effectively on ordering key. When it is necessary to **search** for a **record** on the **basis of** some **other attribute** than the ordering key field, sequential file representation is inadequate.
- **Multiple indexes** can be **maintained** for **each** type of **field** used for searching. Thus, indexing provides much better flexibility.
- An **index file** usually requires **less storage** space than the main file.
- A **binary search** on **sequential file** will **require** accessing of **more blocks**.
- This can be explained with the help of the following example.
- Consider the example of a sequential file with $r = 1024$ **records** of fixed length with **record size $R = 128$ bytes** stored on disk with **block size $B = 2048$ bytes**.

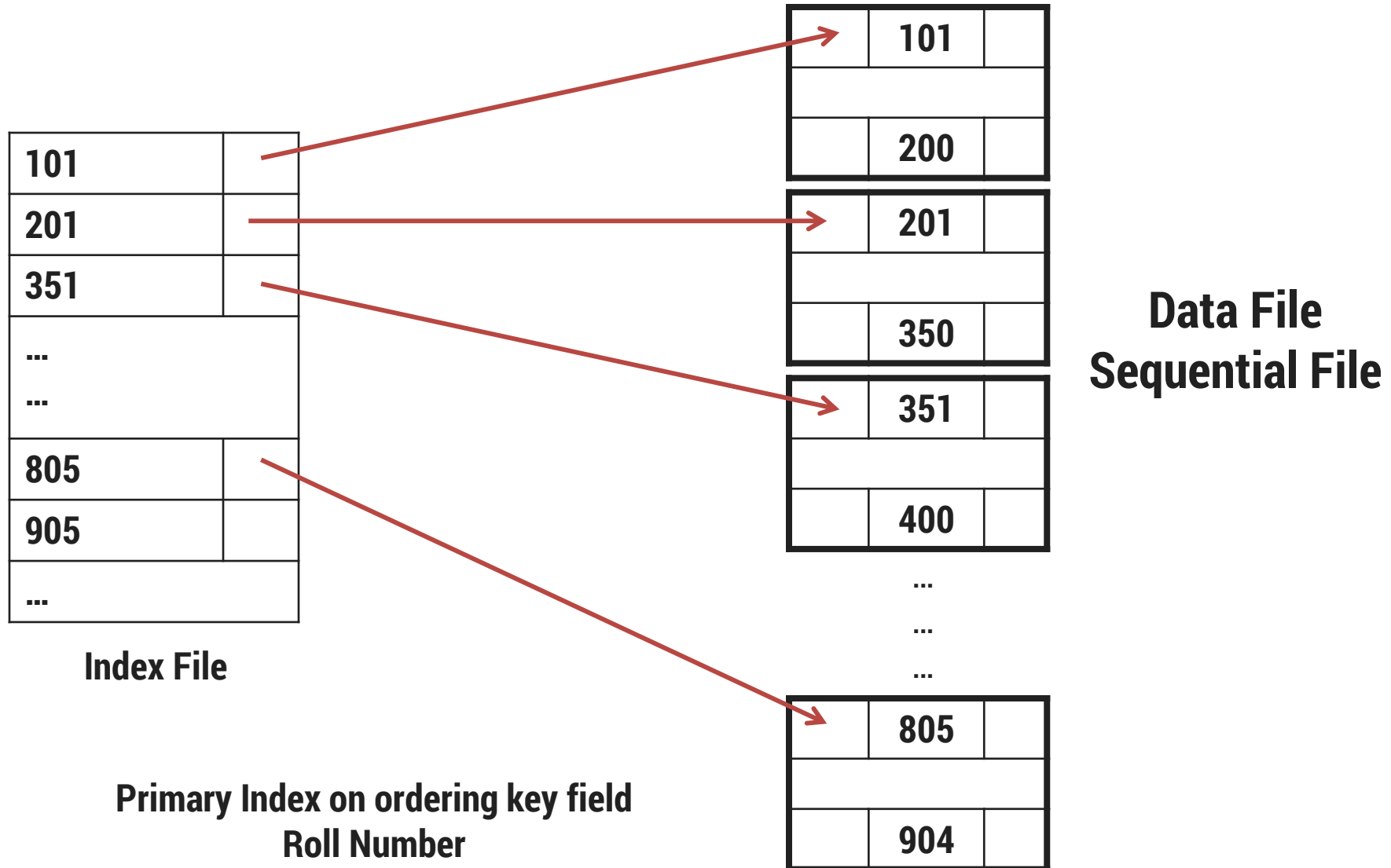
Advantages of Indexing

- **Size of Sequential File**
 - **Number of blocks required to store the file**
 - $(1024 \times 128) / 2048 = 64$
 - **Number of block accesses for searching a record**
 - $\log_2 64 = 6$
- **Size of Index File**
 - Suppose, we want to **construct an index** on a **key field** that is $V = 4$ **bytes** long and the block pointer is $P = 4$ **bytes** long.
 - A **record** of an **index file** needs **8 bytes** per entry.
 - **Total Number of index entries = 1024**
 - **Number of blocks required to store the index file**
 - $(1024 \times 8) / 2048 = 4$
 - **Number of block accesses for searching a record = $\log_2 4 = 2$**

Types of Indexes

- With indexing, **new records** can be **added** at the **end of the main file**. It will not require movement of records as in the case of sequential file.
- **Updation of index file requires fewer block** accesses compare to sequential file
- Types of Indexes:
 1. Primary indexes
 2. Clustering indexes
 3. Secondary indexes

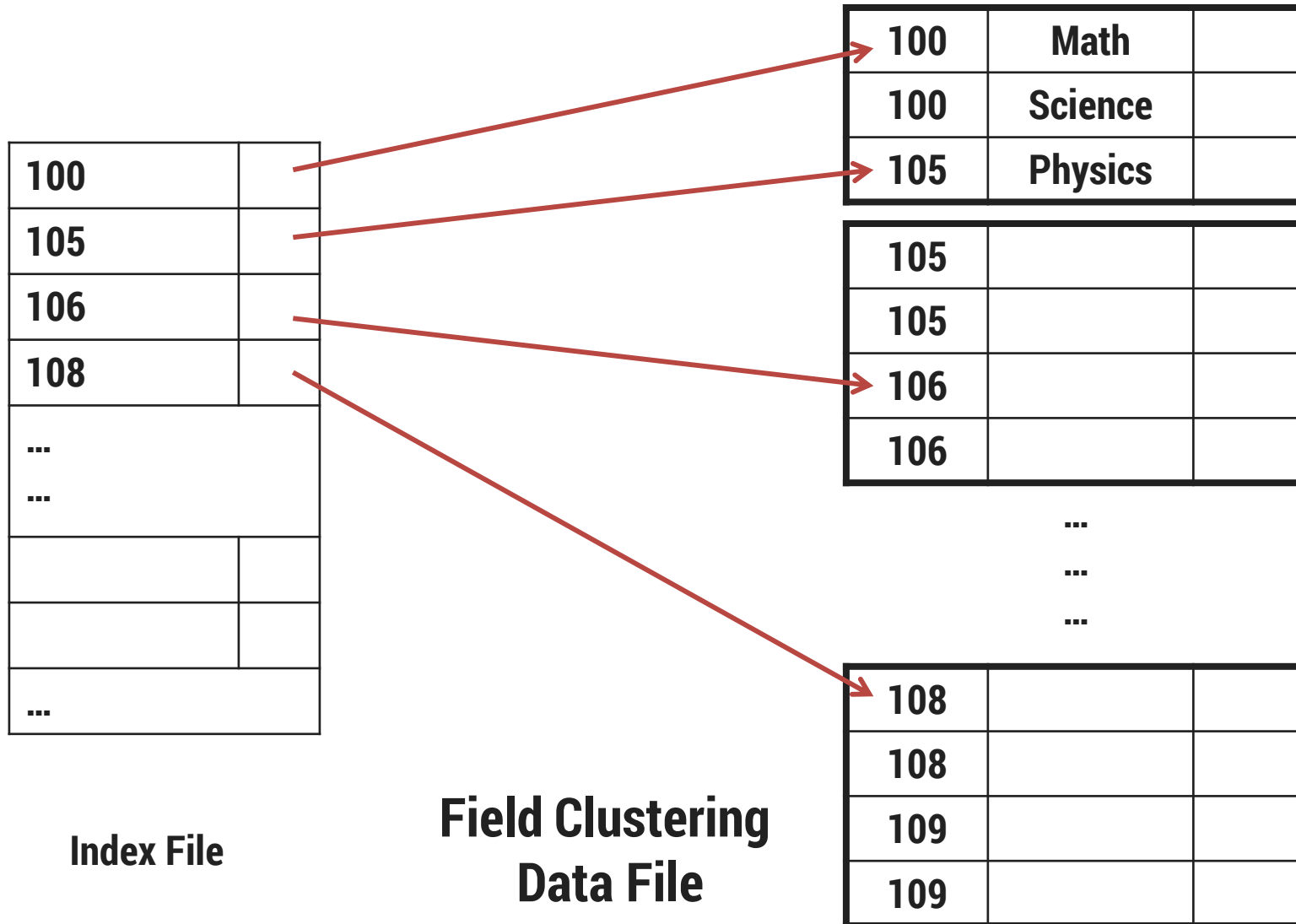
Primary Indexes (Indexed Sequential File)



Primary Indexes (Indexed Sequential File)

- An **indexed sequential** file is **characterized** by
 - Sequential organization (ordered on primary key)
 - Indexed on primary key
- An **indexed sequential** file is both **ordered** and **indexed**.
- **Records** are **organized in sequence** based on a **key field**, known as **primary key**.
- An **index** to the file is **added to support random access**. Each record in the index file consists of two fields: a key field, which is the same as the key field in the main file.
- **Number of records** in the **index file** is equal to the **number of blocks** in the **main file** (data file) and not equal to the number of records in the main file (data file).

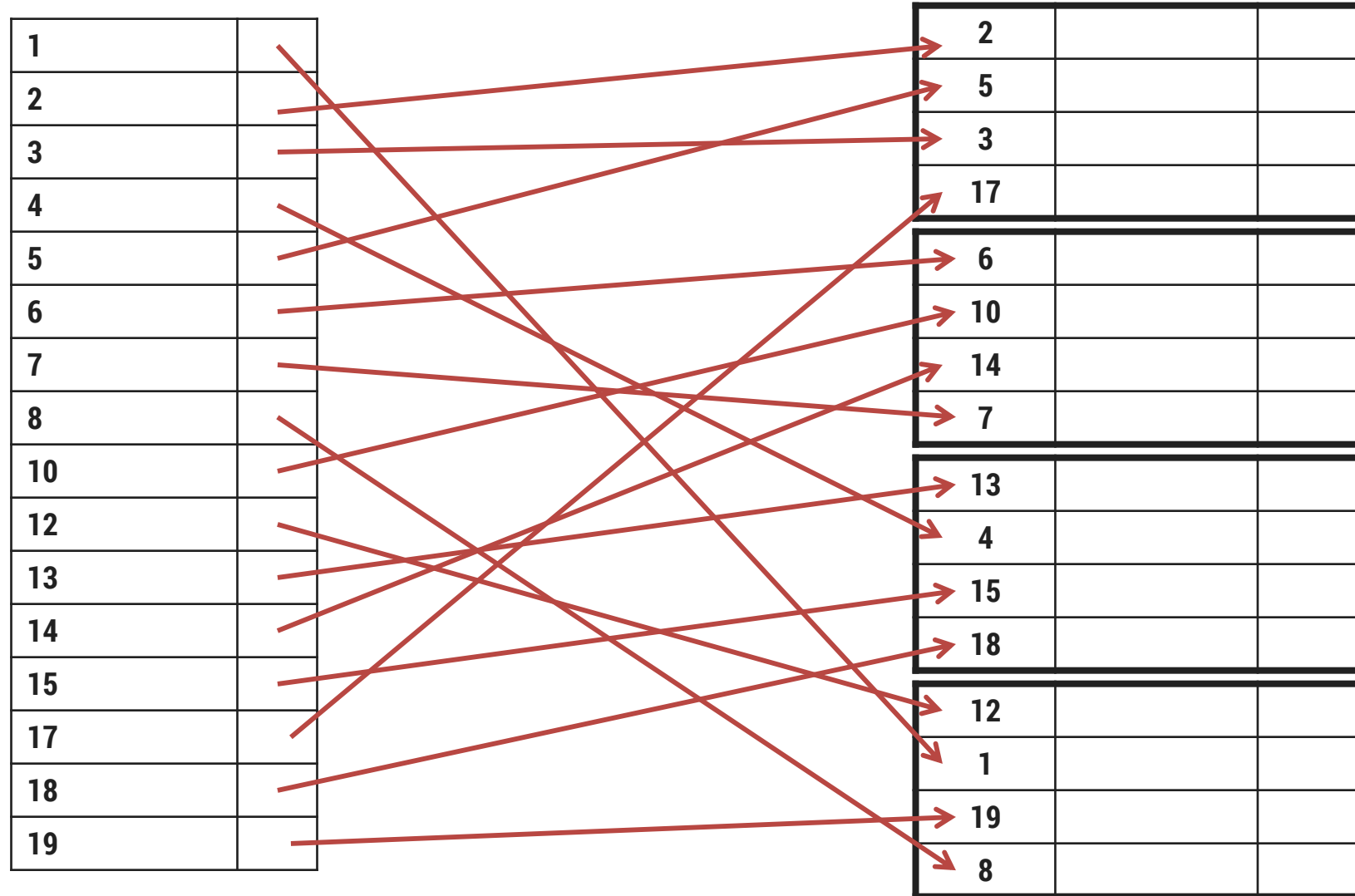
Clustering Indexes



Clustering Indexes

- If **records** of a file are **ordered on a non-key field**, we can create a different type of index known as **clustering index**.
- A non-key field does not have distinct value for each record.
- A Clustering index is also an ordered file with two fields.

Secondary Indexes (Simple Index File)



A secondary index on a non-ordering key field

Secondary Indexes (Simple Index File)

- While the **hashed**, **sequential** and **indexed sequential** files are suitable for operations based on ordering key or the hashed key. Above file organizations are **not suitable** for operations involving a search on a **field other than ordering or hashed key**.
- If searching is required on various keys, **secondary indexes** on these fields must be **maintained**.
- A **secondary index** is an ordered file with **two fields**.
 - Some **non-ordering field** of the data file.
 - A **block pointer**
- There could be **several secondary indexes** for the **same file**.
- One **could use binary search** on index file as entries of the index file are ordered on secondary key field.
- Records of the **data files are not ordered** on secondary key field.

Secondary Indexes (Simple Index File)

- A **secondary index** requires **more storage** space and **longer search time** than does a **primary index**.
- A secondary index file has an **entry for every record** whereas primary index file has an entry for every block in data file.
- There is a **single primary index** file but the **number of secondary indexes** could be quite a few.